



## Project Title: Multi-Class Brain Tumor Classification Using Transfer Learning on MRI Images

### Authors:

| Name              | Student ID |
|-------------------|------------|
| Fahim Alavi Ridoy | 1000060722 |
|                   |            |
|                   |            |
|                   |            |
|                   |            |

### Submission Date:

25/06/2026

**Advanced Training on Semiconductor and ICT Technology**

BRAC University

## Table of Contents

|                                        |    |
|----------------------------------------|----|
| 1. Introduction.....                   | 3  |
| 1.1 Brief Introduction.....            | 3  |
| 1.2 Problem Statement.....             | 3  |
| 1.3 Objectives.....                    | 4  |
| 2. Dataset and Preparation.....        | 4  |
| 2.1 Dataset Description.....           | 4  |
| 2.2 Data Access.....                   | 4  |
| 2.3 Preprocessing.....                 | 4  |
| 3. Methodology.....                    | 5  |
| 3.1 Course Techniques Used.....        | 5  |
| 3.2 Model Architecture.....            | 5  |
| 3.3 Training Strategy.....             | 6  |
| 4. Experiments and Results.....        | 6  |
| 4.1 MLflow Tracking.....               | 6  |
| 4.2 Evaluation Results.....            | 7  |
| 5. Prediction App and Docker.....      | 13 |
| 5.1 Prediction Pipeline.....           | 13 |
| 5.2 Prediction UI.....                 | 14 |
| 6. Repository and Reproducibility..... | 15 |
| 6.1 Repository Structure.....          | 15 |
| 6.2 GitHub Rules.....                  | 15 |
| 6.3 Reproducibility.....               | 16 |
| 7. Limitations and Future Work.....    | 16 |
| 7.1 Limitations.....                   | 16 |
| 7.2 Future Improvements.....           | 16 |
| 8. Conclusion.....                     | 17 |
| 9. References.....                     | 17 |
| Appendix.....                          | 17 |
| A. Final Submission Checklist.....     | 17 |
| B. Final Notes.....                    | 18 |

# 1. Introduction

## 1.1 Brief Introduction

This project operates in the medical imaging domain, focusing on healthcare diagnostics. It builds a deep learning-based multi-class image classification system using transfer learning. The main purpose is to automatically classify brain MRI scans to assist in the early and accurate diagnosis of brain tumors. This project applies deep learning, specifically transfer learning with pretrained CNNs to classify brain MRI scans into one of four categories: glioma, meningioma, pituitary tumor, or no tumor. Four architectures (ResNet50, VGG16, DenseNet121 and EfficientNetB0) are trained. The final system compares four CNN architectures, tracks experiments using MLflow, and provides an interactive, Docker-deployed web application where users can upload MRI scans and receive instant diagnostic predictions.

- Domain: Medical Imaging (Brain Tumor Classification from MRI scans)
- System type: Multi-class image classification using Transfer Learning (pretrained CNNs)
- Purpose: Automate MRI-based brain tumor diagnosis to reduce radiologist workload and inter-observer variability
- Final system: A Dockerized Streamlit app where users upload an MRI image and receive the predicted tumor class with per-class confidence scores, powered by one of four CNN backbones.

## 1.2 Problem Statement

One of the most serious neurological diseases can be considered a brain tumor, so getting diagnosed early and accurately can definitely increase the chance of a patient getting well. Magnetic Resonance Imaging (MRI) is widely recognized as the primary noninvasive technique for brain tumor visualization, yet manual interpretation by radiologists not only takes a lot of time but also varies from one observer to another.

The system receives a 2D brain MRI scan (image) as input and produces a predicted tumor class (glioma, meningioma, pituitary tumor, or no tumor) along with a confidence score and serves it through a Docker-hosted interactive application. This system is designed for radiologists, medical researchers, and healthcare professionals.

Specifically:

- Input: A brain MRI image (JPG/PNG file uploaded by the user)
- Output: Predicted class label (glioma / meningioma / pituitary / no tumor) with confidence score
- Users: Medical researchers, radiologists, and healthcare practitioners seeking decision

support

- Importance: Accurate automated tumor classification can improve early diagnosis and treatment planning, reducing mortality rates

## 1.3 Objectives

The key objectives of this project are:

- Train and compare four pretrained CNN architectures on the brain tumor MRI dataset.
- Track all experiments with MLflow, logging hyperparameters, metrics, and artifacts.
- Build an interactive Streamlit prediction UI supporting MRI image uploads.
- Containerize and serve the app using Docker.
- Ensure full reproducibility through clean code, config files, and a documented GitHub repository.

## 2. Dataset and Preparation

### 2.1 Dataset Description

The project uses the Brain Tumor MRI Dataset by Masoud Nickparvar, sourced from Kaggle. It contains 7,023 MRI images across 4 classes: glioma, meningioma, pituitary tumor, and no tumor. The input type is medical images (MRI scans), and the target/output variable is the categorical tumor class.

Dataset details:

- Name & Source: Brain Tumor MRI Dataset from Kaggle (Masoud Nickparvar)
- Number of samples: 7,023 MRI images
- Classes: 4 classes: glioma, meningioma, pituitary tumor, no tumor
- Input type: Image (MRI scans, resized to 224×224 pixels)
- Target: Tumor class label (4-class classification problem)

### 2.2 Data Access

The dataset is downloaded from Kaggle using the Kaggle API via a fetch script included in the repository. The raw dataset is placed in the local data/ directory folder.

The raw data is downloaded directly into the local data/ folder. The data/ directory is added to .gitignore to ensure no raw datasets are committed to GitHub.

### 2.3 Preprocessing

The following preprocessing steps are applied:

Steps:

- All images resized to 224×224 to match backbone input requirements
- Architecture-specific normalization via each model's Keras preprocess\_input function
- Light augmentation on training data: rotation  $\pm 10^\circ$ , width/height shift 5%, zoom 10%, horizontal flip
- 15% of Training/ split held out as validation set; Testing/ split reserved exclusively for final evaluation
- Class distribution is roughly balanced across four classes within the training split

### 3. Methodology

#### 3.1 Course Techniques Used

The following techniques from the SICIP ML/DL course were applied in this project:

| Technique                         | Where used                           |
|-----------------------------------|--------------------------------------|
| MLflow experiment tracking        | app/app.py, Dockerfile, src/train.py |
| Dockerized prediction app         | src/models.py, src/train.py          |
| Transfer Learning (CNN backbones) | src/models.py, src/train.py          |
| API/UI model serving              | app/app.py, src/predict.py           |
| Evaluation and error analysis     | src/compare_models.py                |

Examples of course techniques:

- Transfer learning (ResNet50, VGG16, DenseNet121, EfficientNetB0)
- CNN-based image classification with ImageNet-pretrained backbones
- Hyperparameter configuration via YAML (configs/config.yaml) with ReduceLROnPlateau and early stopping
- MLflow experiment tracking across four independent training runs
- Docker containerization of the Streamlit prediction application
- Streamlit-based interactive UI for model serving (app/app.py)
- Evaluation with confusion matrices, accuracy/loss metrics, and model comparison (src/compare\_models.py)

#### 3.2 Model Architecture

Four CNN backbones are used as fixed feature extractors with ImageNet pretrained weights:

ResNet50, VGG16, DenseNet121, and EfficientNetB0. Each backbone is loaded with `include_top=False` and its weights frozen. A custom classification head is appended for all backbones:

GlobalAveragePooling2D → Dense(256, relu) → Dropout(0.3) → Dense(4, softmax). Loss: Categorical cross-entropy. Optimizer: Adam (lr=1e-4). Pretrained weights: ImageNet.

### 3.3 Training Strategy

The models were trained using a pure transfer learning approach (frozen backbone, training only the custom head). The batch size was set to 32, and training ran for up to 15 epochs with early stopping (patience=4 on validation loss) and ReduceLROnPlateau learning rate scheduling

Validation strategy: 15% of the Training/ split held out as validation set. LR scheduling: ReduceLROnPlateau (factor=0.5, patience=2). Early stopping: patience=4 on val\_loss. Best model selected by highest test accuracy on the held-out Testing/ split. All runs tracked via MLflow.

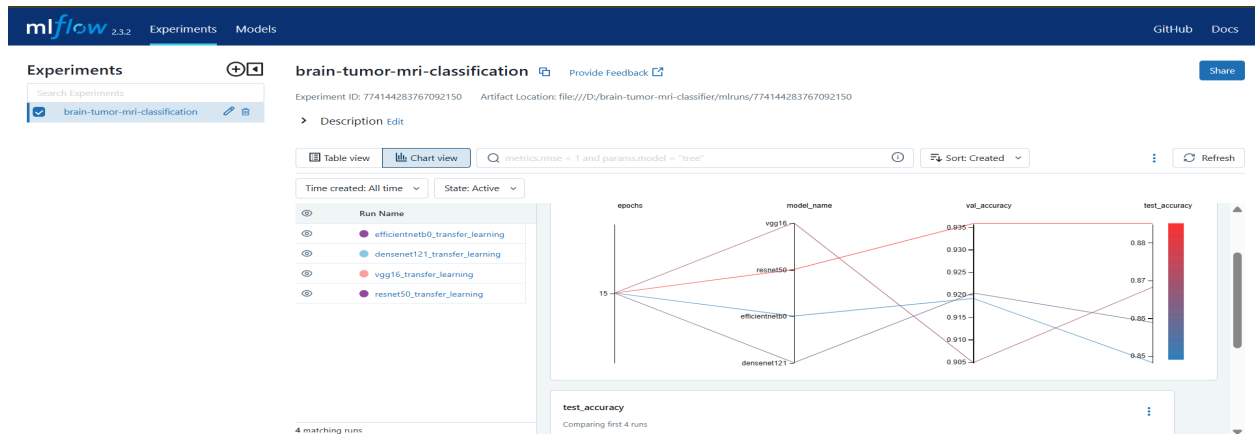
## 4. Experiments and Results

### 4.1 MLflow Tracking

All four training runs (one per backbone) were tracked under the brain-tumor-mri-classification experiment in MLflow.

Logged items:

- Parameters logged: model name, image size, batch size, learning rate, dropout rate, dense units, epochs, early stopping patience.
- Metrics logged: per-epoch training and validation loss/accuracy, final test loss/accuracy.
- Artifacts saved: training curve plots, trained model files, class-index mapping, and confusion matrices.
- Number of runs: 4 independent runs.
- Best run: Selected based on the highest test accuracy on the held-out Testing/ split.



Picture: MLflow\_runs

## 4.2 Evaluation Results

Model comparison results. The table below summarizes test accuracy and loss across all four backbones:

| Model           | Test Accuracy | Test Loss | Validation Accuracy | Validation Loss | Epochs |
|-----------------|---------------|-----------|---------------------|-----------------|--------|
| ResNet50        | <b>88.50%</b> | 0.4302    | <b>93.57%</b>       | 0.1785          | 15     |
| VGG16           | 86.81%        | 0.5208    | 90.48%              | 0.2572          | 15     |
| DenseNet121     | 85.88%        | 0.4352    | 92.02%              | 0.2429          | 15     |
| EfficientNet B0 | 84.81%        | 0.4873    | 91.90%              | 0.2222          | 15     |

Best Model: ResNet50

Classification Report (ResNet50)

| Class                   | Precision | Recall | F1-Score     |
|-------------------------|-----------|--------|--------------|
| Glioma                  | 0.89      | 0.75   | 0.81         |
| Meningioma              | 0.83      | 0.81   | 0.82         |
| No Tumor                | 0.90      | 0.99   | 0.94         |
| Pituitary               | 0.92      | 0.99   | 0.95         |
| <b>Overall Accuracy</b> | -         | -      | <b>88.5%</b> |

#### Key Findings:

- ResNet50 achieved the highest test accuracy (88.50%), outperforming the other three backbones by a margin of 1.69-3.69 percentage points.
- ResNet50 also showed the best generalization, with the highest validation accuracy (93.57%) and lowest test loss (0.4302).
- VGG16 secured second place with 86.81% test accuracy, despite being a simpler architecture.
- DenseNet121 and EfficientNetB0 performed comparably (85.88% and 84.81% respectively), demonstrating that parameter-efficient models can still achieve competitive results on medical imaging tasks.
- All models completed the full 15 epochs without early stopping triggering, suggesting stable training convergence.

The results indicate that deeper residual networks (ResNet50) transfer more effectively to brain tumor classification than classical (VGG16) or compound-scaled architectures (EfficientNetB0) under frozen-backbone transfer learning conditions.

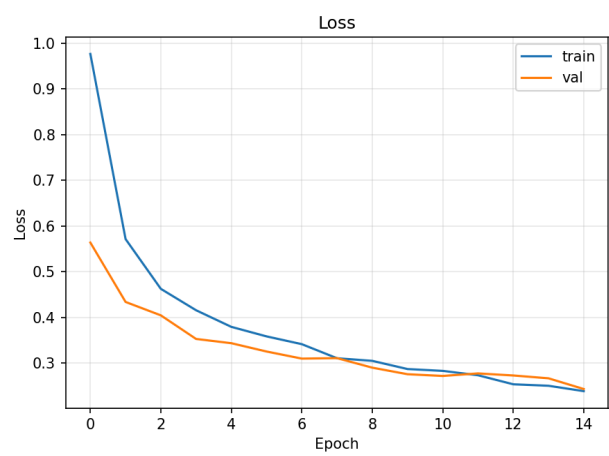
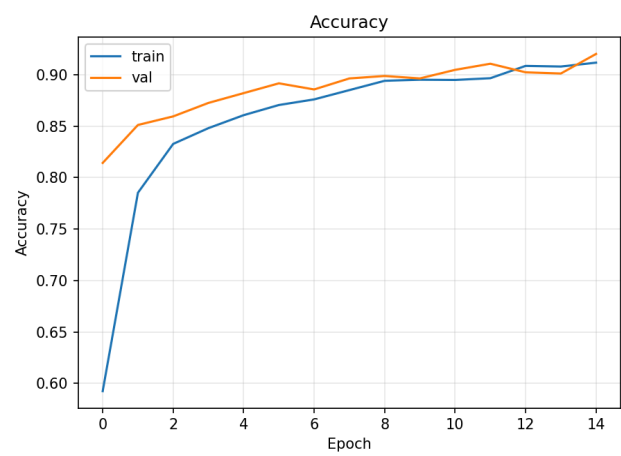
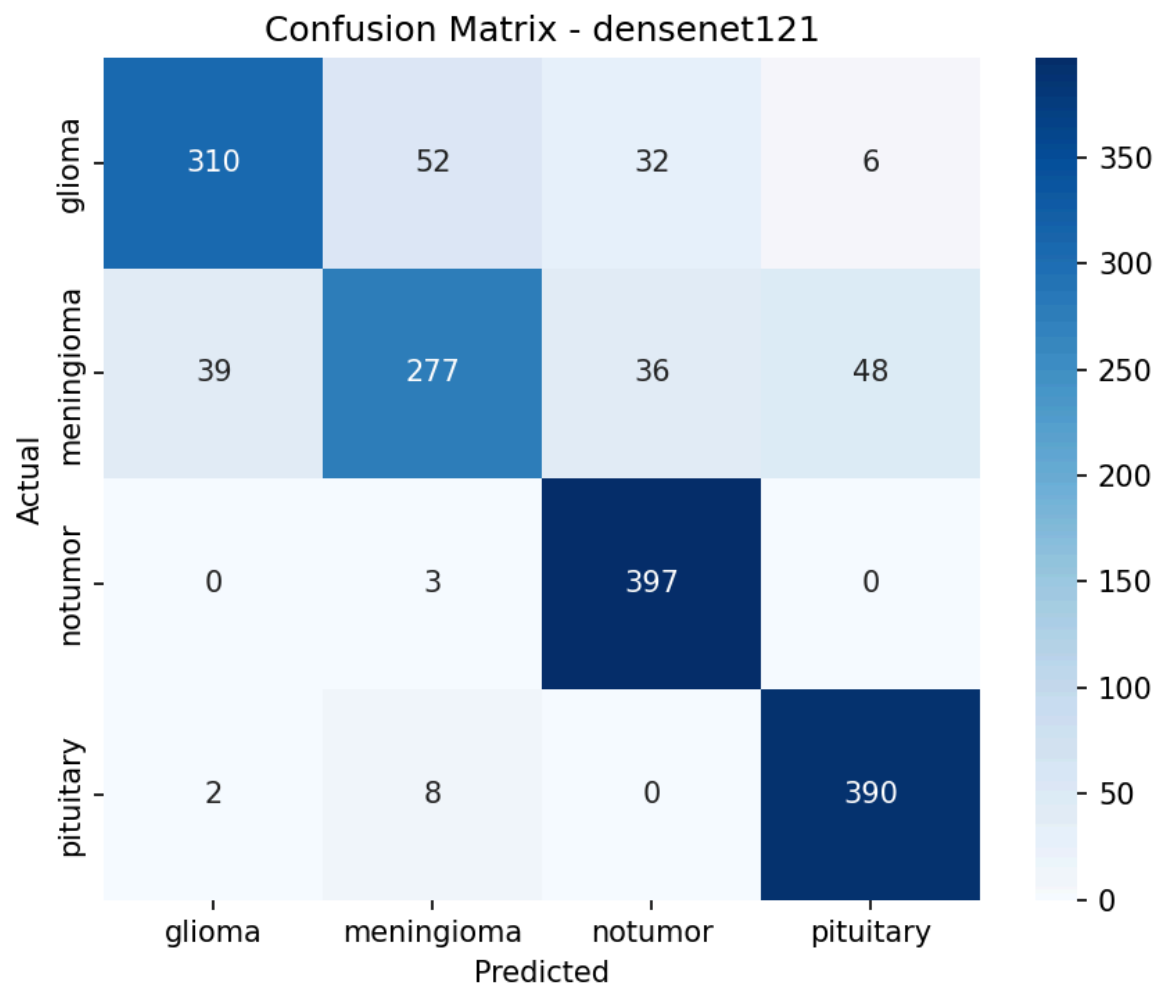
```

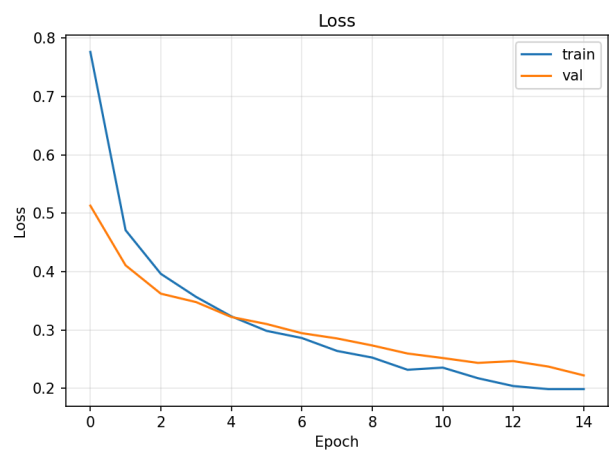
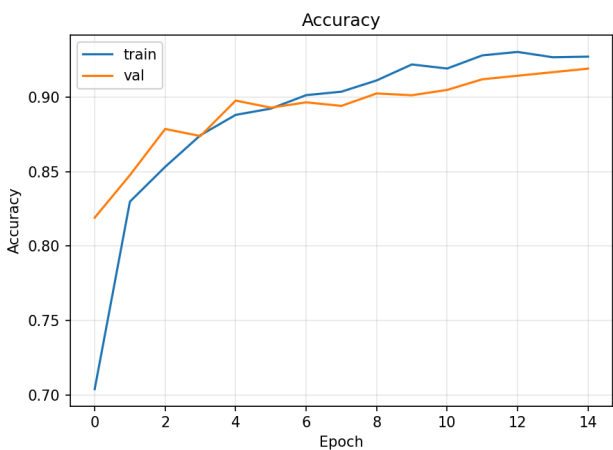
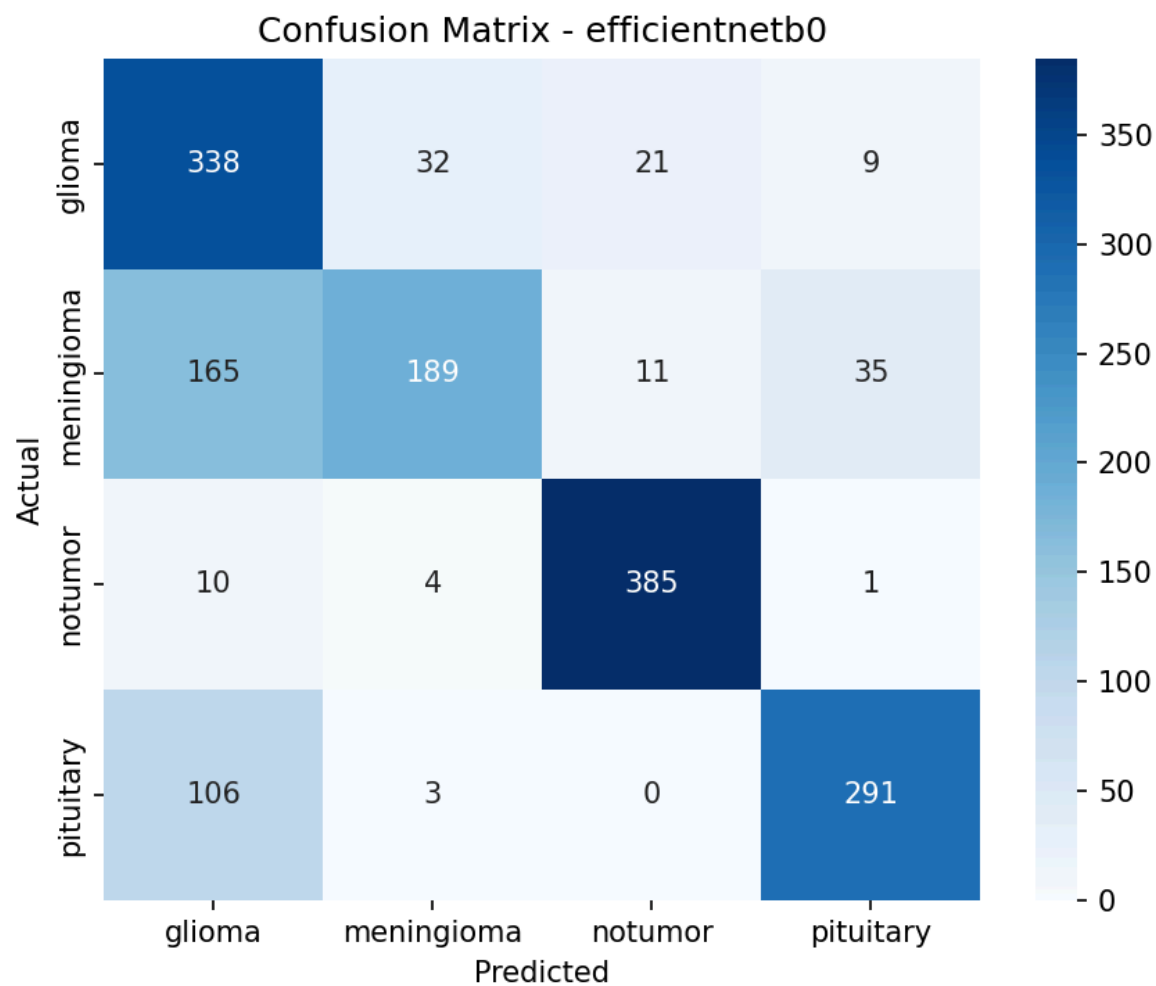
=== Model Comparison (by test accuracy) ===

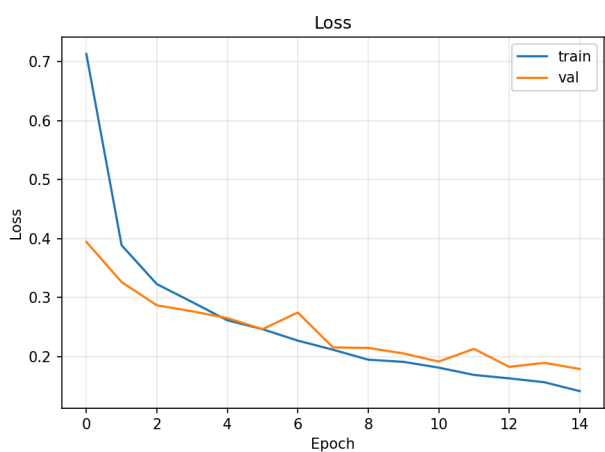
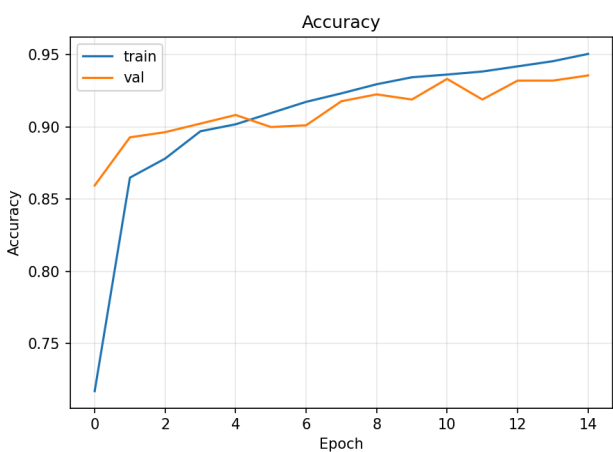
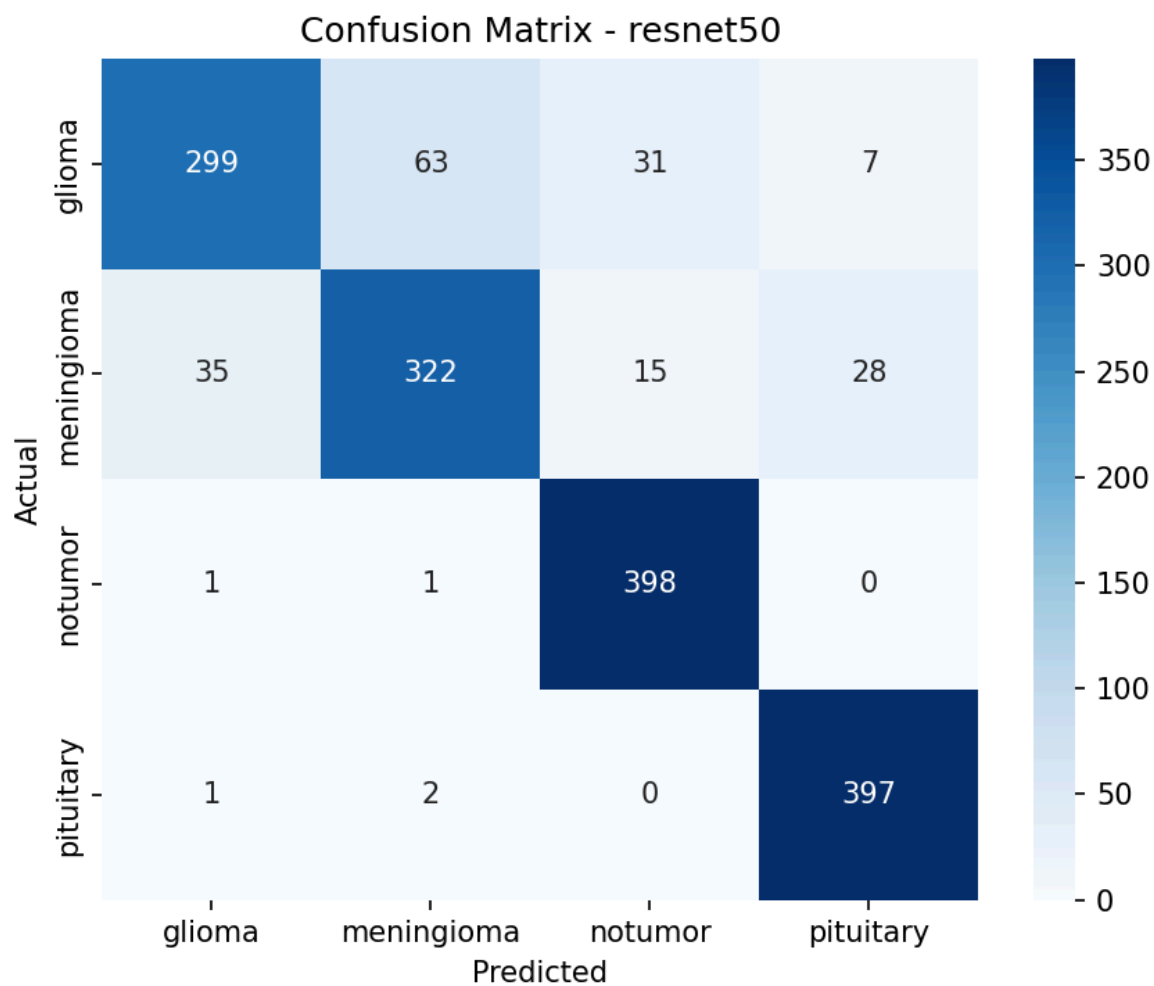
```

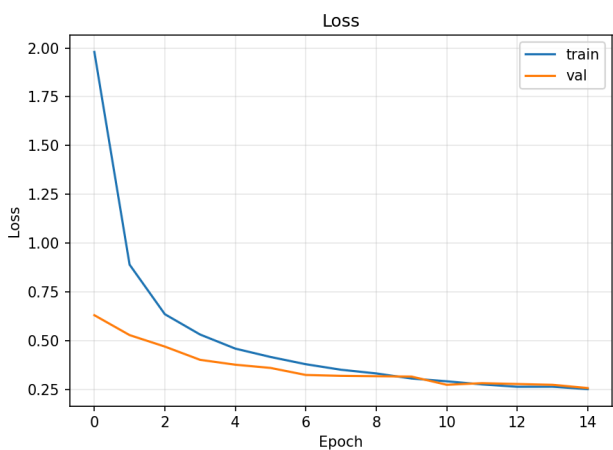
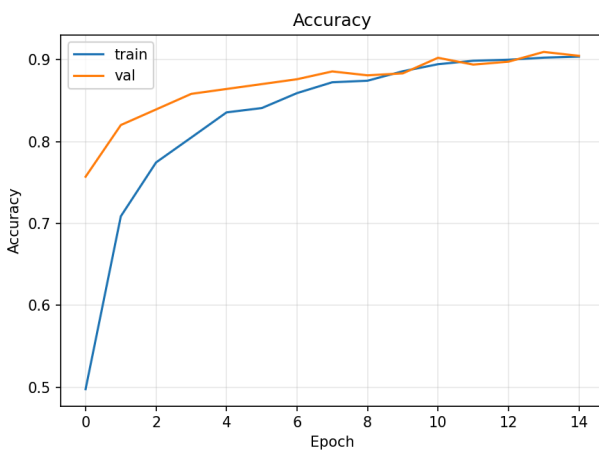
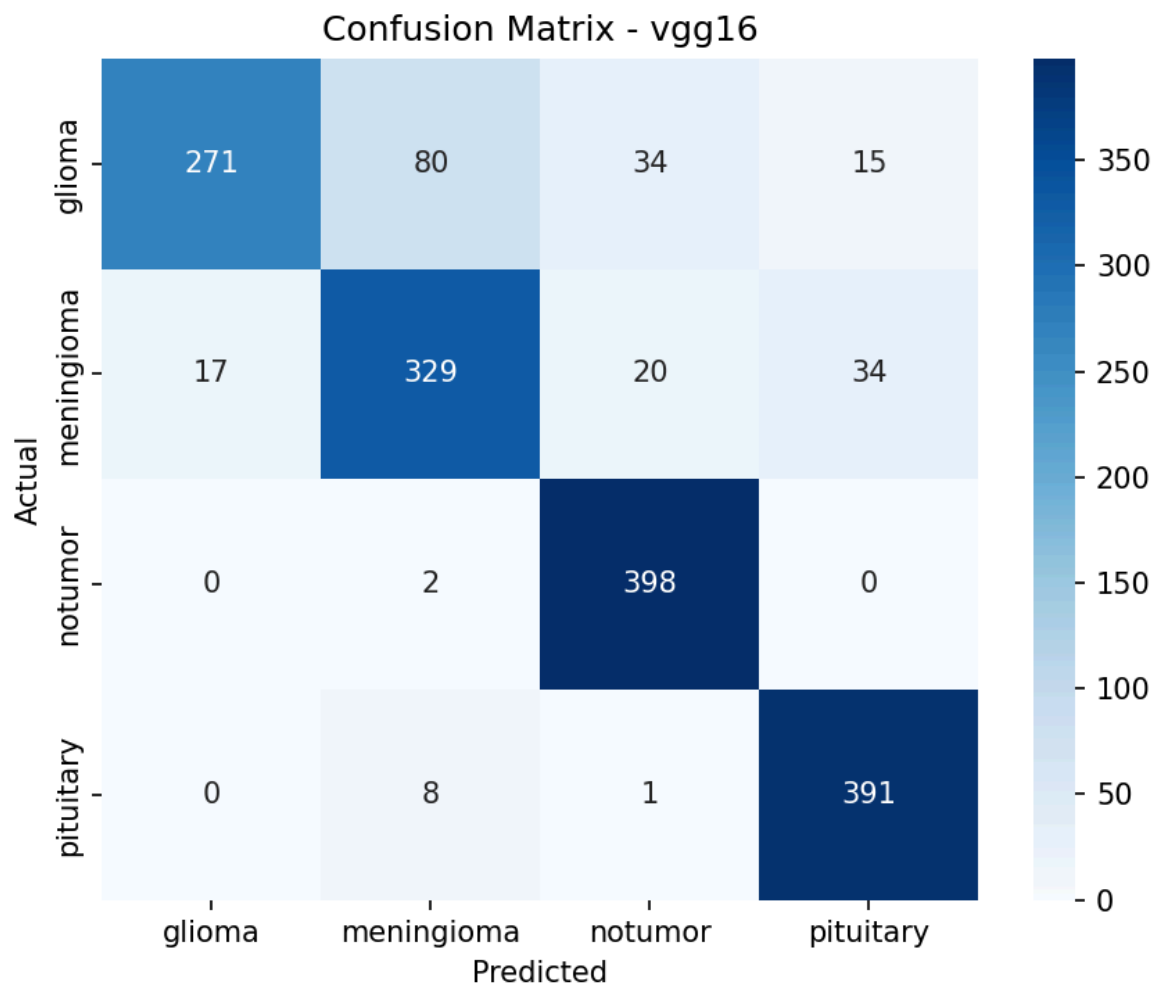
| id | model_name     | test_accuracy | test_loss | val_accuracy | val_loss | epochs | learning_rate | run_                           |
|----|----------------|---------------|-----------|--------------|----------|--------|---------------|--------------------------------|
| ea | resnet50       | 0.885000      | 0.430182  | 0.935714     | 0.178462 | 15     | 0.0001        | 1db4a4ab697046bc8a56f65304749b |
| a6 | vgg16          | 0.868125      | 0.520778  | 0.904762     | 0.257214 | 15     | 0.0001        | a77abea972d24b2aa92cba89f1098d |
| ba | densenet121    | 0.858750      | 0.435159  | 0.920238     | 0.242881 | 15     | 0.0001        | 8737714875be474a9eec836a7cb8f8 |
| f0 | efficientnetb0 | 0.848125      | 0.487330  | 0.919048     | 0.222184 | 15     | 0.0001        | 1578caba2aa044a3bdfc3ae874b60f |

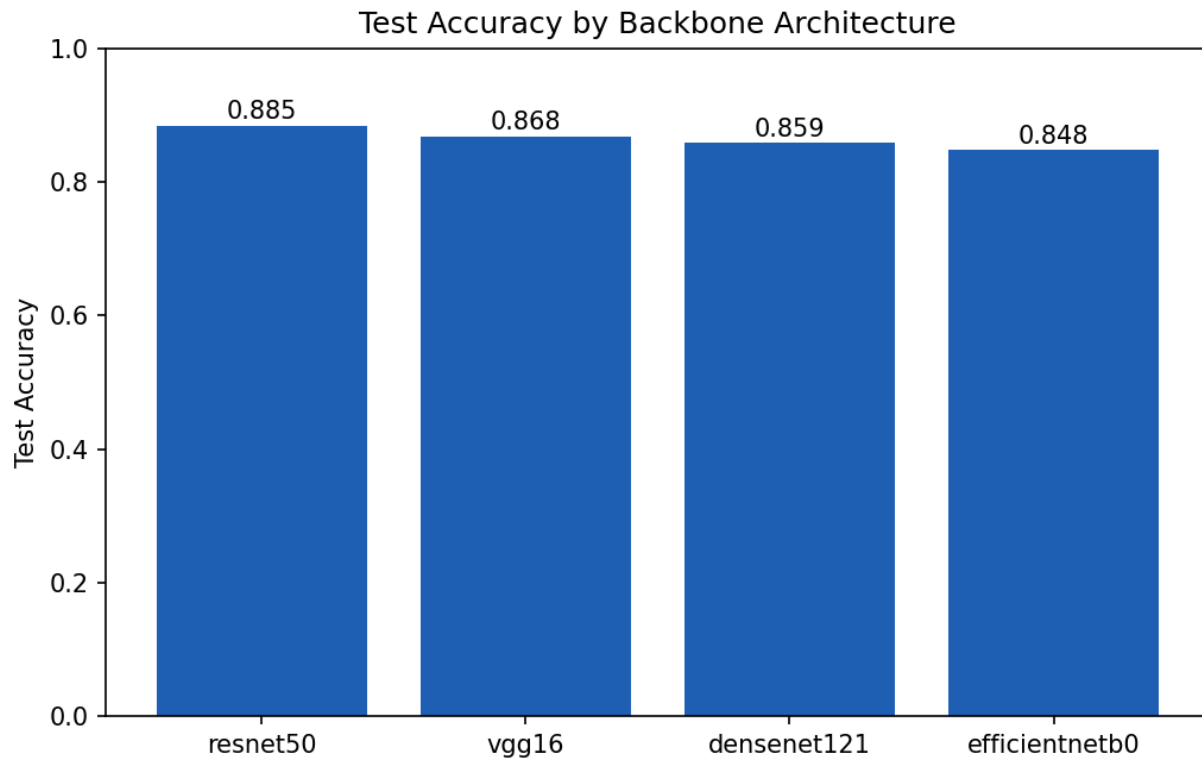
Picture: All Models Results











## 5. Prediction App and Docker

### 5.1 Prediction Pipeline

The prediction pipeline operates as follows:

Pipeline steps:

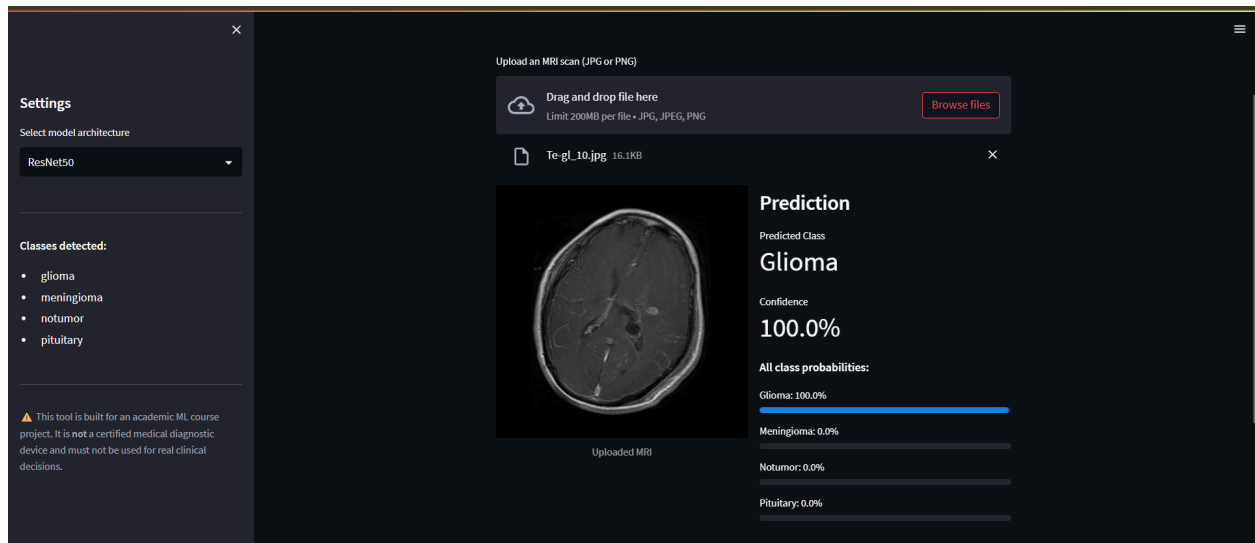
- The user uploads a brain MRI image (JPG/PNG) via the Streamlit interface or provides a path via CLI (src/predict.py)
- The image is resized to 224×224 and normalized using the selected backbone's Keras preprocess\_input function (shared preprocessing logic in src/models.py)
- The frozen backbone extracts features; the classification head produces a 4-class softmax probability distribution
- The predicted class label and per-class confidence scores are displayed to the user in the UI

Both CLI and Streamlit UI share the same preprocessing logic (src/models.py) to guarantee consistent predictions across both paths.

## 5.2 Prediction UI

The prediction UI is built with Streamlit (app/app.py). Users can:

- Select a CNN backbone from a dropdown menu.
- Upload an MRI image file.
- Immediately see the predicted tumor class and a per-class confidence breakdown.

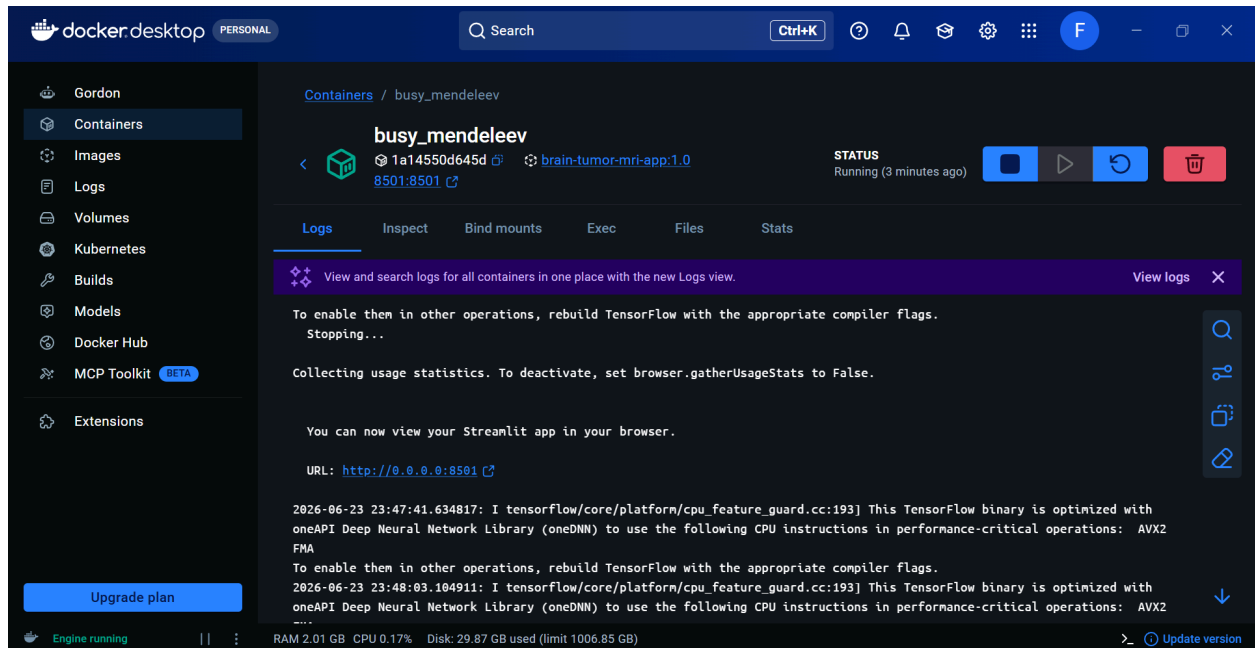


Picture: uploaded MRI image and prediction output

## 5.3 Docker Serving

The Streamlit prediction application is fully containerized using Docker. The repository contains a Dockerfile that bundles the application code, configuration, trained model weights, and class-index mapping.

Build and run instructions are documented in the project README. The container exposes the Streamlit UI on port 8501.



Picture: Docker container terminal output confirming the app is running

## 6. Repository and Reproducibility

### 6.1 Repository Structure

The repository follows a clean, reproducible structure:

Key directories and files:

- README.md, final\_report.md, requirements.txt, .gitignore, Dockerfile
- src/: Core training, evaluation, and prediction scripts.
- app/: Streamlit UI application.
- configs/: Configuration files (e.g., config.yaml).
- screenshots/: Visual evidence for the report.
- mlruns/: MLflow experiment tracking data.
- models/: Saved model weights.
- data/: Local dataset directory (ignored in Git).

### 6.2 GitHub Rules

The following are excluded from the GitHub repository:

- Raw datasets and downloaded data (data/ directory)
- Virtual environments (.venv/, env/)
- Cache files (\_\_pycache\_\_/, .pyc files)
- Private credentials or API keys
- Large generated artifacts (unless required for evaluation)

- Note: mlruns/ is intentionally NOT ignored. MLflow runs are committed as evidence of experiment tracking

Crucially, the mlruns/ directory is committed and not ignored, serving as verifiable evidence of training and experiment tracking.

### 6.3 Reproducibility

A reviewer can reproduce the full project from a fresh clone by following these steps in order:

Expected steps:

- Install dependencies from requirements.txt
- Fetch the dataset into data/
- Run training: python src/train.py for each backbone (tracked via MLflow)
- Review MLflow experiment runs: mlflow ui
- Compare models: python src/compare\_models.py (generates CSV and bar chart)
- Build and run the Docker prediction app per README instructions

Full step-by-step commands are provided in the project README.

## 7. Limitations and Future Work

### 7.1 Limitations

Known limitations of this project:

Limitations:

- Only 2D slice-level classification is performed; no 3D volumetric context is used
- The notumor class originates from a different source dataset (Br35H), introducing a possible domain-shift shortcut the model could exploit
- All backbones trained with frozen weights only; deeper fine-tuning not exhaustively explored due to CPU-only constraints
- GPU acceleration via TensorFlow-DirectML encountered an unresolved plugin bug and could not be used
- No external validation on scans from a different hospital or scanner so real-world generalization is unverified.
- This system is a research/educational prototype, not a validated clinical decision-support tool

### 7.2 Future Improvements

Planned improvements for future iterations:

Ideas:

- Fine-tune the last N convolutional layers of each backbone (already supported via fine\_tune\_last\_n\_layers in configs/config.yaml)

- Add Grad-CAM or saliency-map visualizations for interpretable clinical review
- Conduct systematic hyperparameter search (learning rate, dropout, dense layer size) with MLflow tracking each trial
- Extend Docker image to expose a REST API (FastAPI) alongside the Streamlit UI for programmatic integration
- Validate on an external, independently sourced MRI dataset to assess true generalization

## 8. Conclusion

This project demonstrates a complete, reproducible deep learning pipeline for multi-class brain tumor classification using transfer learning. Four CNN architectures (ResNet50, VGG16, DenseNet121, EfficientNetB0) were trained and compared under identical conditions with full MLflow experiment tracking.

Key outcomes:

- Problem solved: Automated classification of brain MRI scans into four tumor categories (glioma, meningioma, pituitary, no tumor)
- System built: Transfer learning pipeline with four pretrained CNN backbones, tracked via MLflow, served via Dockerized Streamlit app
- Main finding: All four backbones achieved competitive accuracy on the held-out test set; detailed comparison available in artifacts/model\_comparison.csv
- Prediction app: The Dockerized Streamlit application demonstrates end-to-end functionality where users upload MRI images and receive instant tumor class predictions with confidence scores
- Learning outcomes: Practical experience with transfer learning, experiment tracking (MLflow), containerization (Docker), and building user-facing ML applications

## 9. References

1. Nickparvar, M. *Brain Tumor MRI Dataset*. Kaggle.  
<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>
2. He, K. et al. (2016). *Deep Residual Learning for Image Recognition* (ResNet).
3. Simonyan, K. & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition* (VGG16).
4. Huang, G. et al. (2017). *Densely Connected Convolutional Networks* (DenseNet).
5. Tan, M. & Le, Q. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*.

## Appendix

### A. Final Submission Checklist

- ☒ Selected one approved project domain.

- ☒ Public GitHub repository is accessible.
- ☒ Dataset is not committed to GitHub.
- ☒ Data can be downloaded into data/.
- ☒ Training uses MLflow.
- ☒ mlruns/ is included and not ignored.
- ☒ At least two MLflow runs are logged.
- ☒ Report includes MLflow screenshot.
- ☒ Prediction app supports sample upload/input.
- ☒ Prediction app is served using Docker.
- ☒ Report includes Docker app screenshot.
- ☒ Course techniques used are clearly mentioned.
- ☒ Deadline is respected: **25 June 2026**.

## B. Final Notes

This project Multi-Class Brain Tumor Classification Using Transfer Learning on MRI Images was completed and submitted on 25 June 2026, meeting the SICIP certification deadline.

Summary of completed work:

- Four CNN backbones (ResNet50, VGG16, DenseNet121, EfficientNetB0) were trained as frozen feature extractors on the Brain Tumor MRI Dataset (7,023 images, 4 classes), with all runs tracked in MLflow under the brain-tumor-mri-classification experiment.
- ResNet50 was selected as the best model, achieving 88.50% test accuracy and 93.57% validation accuracy, outperforming VGG16 (86.81%), DenseNet121 (85.88%), and EfficientNetB0 (84.81%).
- A Streamlit prediction app (app/app.py) was built supporting backbone selection, MRI image upload, and per-class confidence display, sharing preprocessing logic with the CLI predictor (src/predict.py).
- The app was fully containerized with Docker, exposing the Streamlit UI on port 8501, with a verified terminal screenshot confirming the container runs successfully.
- The repository is structured for reproducibility, with mlruns/ deliberately committed as evidence of experiment tracking and data/ excluded per .gitignore rules.